

Lista4

July 3, 2025

0.1 PUC-Rio

0.1.1 Departamento de Engenharia Elétrica

0.1.2 Lista de Exercícios - Echo State Network

```
[33]: """
      This examples is not intended to be optimized. Its purpose is to show how to
      ↪handle
      big datasets with multiple sequences. The accuracy should be around 10%.
      """

import re
import torch
import torch.nn as nn
from torch.nn import functional as F
from torch.nn.utils.rnn import PackedSequence
import torch.sparse

def apply_permutation(tensor, permutation, dim=1):
    # type: (Tensor, Tensor, int) -> Tensor
    return tensor.index_select(dim, permutation)

class Reservoir(nn.Module):

    def __init__(self, mode, input_size, hidden_size, num_layers, leaking_rate,
                  spectral_radius, w_ih_scale,
                  density, bias=True, batch_first=False):
        super(Reservoir, self).__init__()
        self.mode = mode
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.leaking_rate = leaking_rate
        self.spectral_radius = spectral_radius
        self.w_ih_scale = w_ih_scale
        self.density = density
```

```

self.bias = bias
self.batch_first = batch_first

self._all_weights = []
for layer in range(num_layers):
    layer_input_size = input_size if layer == 0 else hidden_size

    w_ih = nn.Parameter(torch.Tensor(hidden_size, layer_input_size))
    w_hh = nn.Parameter(torch.Tensor(hidden_size, hidden_size))
    b_ih = nn.Parameter(torch.Tensor(hidden_size))
    layer_params = (w_ih, w_hh, b_ih)

    param_names = ['weight_ih_1{}{}'.format(layer, layer_input_size), 'weight_hh_1{}{}'.format(layer, hidden_size)]
    if bias:
        param_names += ['bias_ih_1{}{}'.format(layer, hidden_size)]
    param_names = [x.format(layer, ' ') for x in param_names]

    for name, param in zip(param_names, layer_params):
        setattr(self, name, param)
    self._all_weights.append(param_names)

self.reset_parameters()

def _apply(self, fn):
    ret = super(Reservoir, self)._apply(fn)
    return ret

def reset_parameters(self):
    weight_dict = self.state_dict()
    for key, value in weight_dict.items():
        if key == 'weight_ih_10':
            nn.init.uniform_(value, -1, 1)
            value *= self.w_ih_scale[1:]
        elif re.fullmatch('weight_ih_1[~0]*', key):
            nn.init.uniform_(value, -1, 1)
        elif re.fullmatch('bias_ih_1[0-9]*', key):
            nn.init.uniform_(value, -1, 1)
            value *= self.w_ih_scale[0]
        elif re.fullmatch('weight_hh_1[0-9]*', key):
            w_hh = torch.Tensor(self.hidden_size * self.hidden_size)
            w_hh.uniform_(-1, 1)
            if self.density < 1:
                zero_weights = torch.randperm(
                    int(self.hidden_size * self.hidden_size))
                zero_weights = zero_weights[
                    :int(
                        self.hidden_size * self.hidden_size * (

```

```

        1 - self.density)))

        w_hh[zero_weights] = 0
        w_hh = w_hh.view(self.hidden_size, self.hidden_size)
        abs_eigs = torch.abs(torch.linalg.eigvals(w_hh))
        weight_dict[key] = w_hh * (self.spectral_radius / torch.
↪max(abs_eigs))

        self.load_state_dict(weight_dict)

    def check_input(self, input, batch_sizes):
        # type: (Tensor, Optional[Tensor]) -> None
        expected_input_dim = 2 if batch_sizes is not None else 3
        if input.dim() != expected_input_dim:
            raise RuntimeError(
                'input must have {} dimensions, got {}'.format(
                    expected_input_dim, input.dim()))
        if self.input_size != input.size(-1):
            raise RuntimeError(
                'input.size(-1) must be equal to input_size. Expected {}, got_
↪{}'.format(
                    self.input_size, input.size(-1)))

    def get_expected_hidden_size(self, input, batch_sizes):
        # type: (Tensor, Optional[Tensor]) -> Tuple[int, int, int]
        if batch_sizes is not None:
            mini_batch = batch_sizes[0]
            mini_batch = int(mini_batch)
        else:
            mini_batch = input.size(0) if self.batch_first else input.size(1)
        expected_hidden_size = (self.num_layers, mini_batch, self.hidden_size)
        return expected_hidden_size

    def check_hidden_size(self, hx, expected_hidden_size, msg='Expected hidden_
↪size {}, got {}'):
        # type: (Tensor, Tuple[int, int, int], str) -> None
        if hx.size() != expected_hidden_size:
            raise RuntimeError(msg.format(expected_hidden_size, tuple(hx.
↪size()))))

    def check_forward_args(self, input, hidden, batch_sizes):
        # type: (Tensor, Tensor, Optional[Tensor]) -> None
        self.check_input(input, batch_sizes)
        expected_hidden_size = self.get_expected_hidden_size(input, batch_sizes)

        self.check_hidden_size(hidden, expected_hidden_size)

    def permute_hidden(self, hx, permutation):

```

```

    # type: (Tensor, Optional[Tensor]) -> Tensor
    if permutation is None:
        return hx
    return apply_permutation(hx, permutation)

def forward(self, input, hx=None):
    is_packed = isinstance(input, PackedSequence)
    if is_packed:
        input, batch_sizes, sorted_indices, unsorted_indices = input
        max_batch_size = int(batch_sizes[0])
    else:
        batch_sizes = None
        max_batch_size = input.size(0) if self.batch_first else input.
↪size(1)
        sorted_indices = None
        unsorted_indices = None

    if hx is None:
        hx = input.new_zeros(self.num_layers, max_batch_size,
                             self.hidden_size, requires_grad=False)
    else:
        # Each batch of the hidden state should match the input sequence.
↪that
        # the user believes he/she is passing in.
        hx = self.permute_hidden(hx, sorted_indices)

    flat_weight = None

    self.check_forward_args(input, hx, batch_sizes)
    func = AutogradReservoir(
        self.mode,
        self.input_size,
        self.hidden_size,
        num_layers=self.num_layers,
        batch_first=self.batch_first,
        train=self.training,
        variable_length=is_packed,
        flat_weight=flat_weight,
        leaking_rate=self.leaking_rate
    )
    output, hidden = func(input, self.all_weights, hx, batch_sizes)
    if is_packed:
        output = PackedSequence(output, batch_sizes, sorted_indices,
↪unsorted_indices)
        return output, self.permute_hidden(hidden, unsorted_indices)

def extra_repr(self):

```

```

s = '({input_size}, {hidden_size}'
if self.num_layers != 1:
    s += ', num_layers={num_layers}'
if self.bias is not True:
    s += ', bias={bias}'
if self.batch_first is not False:
    s += ', batch_first={batch_first}'
s += ')'
return s.format(**self.__dict__)

def __setstate__(self, d):
    super(Reservoir, self).__setstate__(d)
    self.__dict__.setdefault('_data_ptrs', [])
    if 'all_weights' in d:
        self._all_weights = d['all_weights']
    if isinstance(self._all_weights[0][0], str):
        return
    num_layers = self.num_layers
    self._all_weights = []
    for layer in range(num_layers):
        weights = ['weight_ih_1{}{}'.format(layer, i), 'weight_hh_1{}{}'.format(layer, i), 'bias_ih_1{}{}'.format(layer, i)]
        weights = [x.format(layer) for x in weights]
        if self.bias:
            self._all_weights += [weights]
        else:
            self._all_weights += [weights[:2]]

@property
def all_weights(self):
    return [[getattr(self, weight) for weight in weights] for weights in
            self._all_weights]

def AutogradReservoir(mode, input_size, hidden_size, num_layers=1,
                      batch_first=False, train=True,
                      batch_sizes=None, variable_length=False, flat_weight=None,
                      leaking_rate=1):
    if mode == 'RES_TANH':
        cell = ResTanhCell
    elif mode == 'RES_RELU':
        cell = ResReLUCell
    elif mode == 'RES_ID':
        cell = ResIdCell

    if variable_length:
        layer = (VariableRecurrent(cell, leaking_rate),)
    else:

```

```

        layer = (Recurrent(cell, leaking_rate),)

    func = StackedRNN(layer,
                      num_layers,
                      False,
                      train=train)

    def forward(input, weight, hidden, batch_sizes):
        if batch_first and batch_sizes is None:
            input = input.transpose(0, 1)

        nexth, output = func(input, hidden, weight, batch_sizes)

        if batch_first and not variable_length:
            output = output.transpose(0, 1)

        return output, nexth

    return forward

def Recurrent(inner, leaking_rate):
    def forward(input, hidden, weight, batch_sizes):
        output = []
        steps = range(input.size(0))
        for i in steps:
            hidden = inner(input[i], hidden, leaking_rate, *weight)
            # hack to handle LSTM
            output.append(hidden[0] if isinstance(hidden, tuple) else hidden)

        output = torch.cat(output, 0).view(input.size(0), *output[0].size())

        return hidden, output

    return forward

def VariableRecurrent(inner, leaking_rate):
    def forward(input, hidden, weight, batch_sizes):
        output = []
        input_offset = 0
        last_batch_size = batch_sizes[0]
        hiddens = []
        flat_hidden = not isinstance(hidden, tuple)
        if flat_hidden:
            hidden = (hidden,)
        for batch_size in batch_sizes:

```

```

        step_input = input[input_offset:input_offset + batch_size]
        input_offset += batch_size

        dec = last_batch_size - batch_size
        if dec > 0:
            hiddens.append(tuple(h[-dec:] for h in hidden))
            hidden = tuple(h[:-dec] for h in hidden)
        last_batch_size = batch_size

        if flat_hidden:
            hidden = (inner(step_input, hidden[0], leaking_rate, *weight),)
        else:
            hidden = inner(step_input, hidden, leaking_rate, *weight)

        output.append(hidden[0])
        hiddens.append(hidden)
        hiddens.reverse()

        hidden = tuple(torch.cat(h, 0) for h in zip(*hiddens))
        assert hidden[0].size(0) == batch_sizes[0]
        if flat_hidden:
            hidden = hidden[0]
        output = torch.cat(output, 0)

    return hidden, output

return forward

def StackedRNN(inners, num_layers, lstm=False, train=True):
    num_directions = len(inners)
    total_layers = num_layers * num_directions

    def forward(input, hidden, weight, batch_sizes):
        assert (len(weight) == total_layers)
        next_hidden = []
        all_layers_output = []

        for i in range(num_layers):
            all_output = []
            for j, inner in enumerate(inners):
                l = i * num_directions + j

                hy, output = inner(input, hidden[l], weight[l], batch_sizes)
                next_hidden.append(hy)
                all_output.append(output)

```

```

        input = torch.cat(all_output, input.dim() - 1)
        all_layers_output.append(input)

    all_layers_output = torch.cat(all_layers_output, -1)
    next_hidden = torch.cat(next_hidden, 0).view(
        total_layers, *next_hidden[0].size())

    return next_hidden, all_layers_output

return forward

def ResTanhCell(input, hidden, leaking_rate, w_ih, w_hh, b_ih=None):
    hy_ = torch.tanh(F.linear(input, w_ih, b_ih) + F.linear(hidden, w_hh))
    hy = (1 - leaking_rate) * hidden + leaking_rate * hy_
    return hy

def ResReLUCell(input, hidden, leaking_rate, w_ih, w_hh, b_ih=None):
    hy_ = F.relu(F.linear(input, w_ih, b_ih) + F.linear(hidden, w_hh))
    hy = (1 - leaking_rate) * hidden + leaking_rate * hy_
    return hy

def ResIdCell(input, hidden, leaking_rate, w_ih, w_hh, b_ih=None):
    hy_ = F.linear(input, w_ih, b_ih) + F.linear(hidden, w_hh)
    hy = (1 - leaking_rate) * hidden + leaking_rate * hy_
    return hy

```

```
[34]: import torch
```

```

def prepare_target(target, seq_lengths, washout, batch_first=False):
    """ Preprocess target for offline training.

    Args:
        target (seq_len, batch, output_size): tensor containing
            the features of the target sequence.
        seq_lengths: list of lengths of each sequence in the batch.
        washout: number of initial timesteps during which output of the
            reservoir is not forwarded to the readout. One value per sample.
        batch_first: If ``True``, then the input and output tensors are provided
            as (batch, seq, feature). Default: ``False``

    Returns:
        tensor containing the features of the batch's sequences rolled out along
        one axis, minus the washouts and the padded values.

```



```

"""

if batch_first:
    target = target.transpose(0, 1)
    n_sequences = target.size(1)
    target_dim = target.size(2)
    train_len = sum(torch.tensor(seq_lengths) - torch.tensor(washout)).item()

    new_target = torch.zeros(train_len, target_dim, device=target.device)

    idx = 0
    for s in range(n_sequences):
        batch_len = seq_lengths[s] - washout[s]
        new_target[idx:idx + batch_len, :] = target[washout[s]:seq_lengths[s], :]
    ↪s, :]
        idx += batch_len

    return new_target

def washout_tensor(tensor, washout, seq_lengths, bidirectional=False, ↪
    ↪batch_first=False):
    tensor = tensor.transpose(0, 1) if batch_first else tensor.clone()
    if type(seq_lengths) == list:
        seq_lengths = seq_lengths.copy()
    if type(seq_lengths) == torch.Tensor:
        seq_lengths = seq_lengths.clone()

    for b in range(tensor.size(1)):
        if washout[b] > 0:
            tmp = tensor[washout[b]:seq_lengths[b], b].clone()
            tensor[:seq_lengths[b] - washout[b], b] = tmp
            tensor[seq_lengths[b] - washout[b]:, b] = 0
            seq_lengths[b] -= washout[b]

            if bidirectional:
                tensor[seq_lengths[b] - washout[b]:, b] = 0
                seq_lengths[b] -= washout[b]

    if type(seq_lengths) == list:
        max_len = max(seq_lengths)
    else:
        max_len = max(seq_lengths).item()

    return tensor[:max_len], seq_lengths

```

```
[35]: import torch
import torch.nn as nn
#from torch.nn.utils.rnn import PackedSequence, pad_packed_sequence
#from .reservoir import Reservoir
#from ..utils import washout_tensor

class ESN(nn.Module):
    """ Applies an Echo State Network to an input sequence. Multi-layer Echo
    State Network is based on paper
    Deep Echo State Network (DeepESN): A Brief Survey - Gallicchio, Micheli 2017

    Args:
        input_size: The number of expected features in the input x.
        hidden_size: The number of features in the hidden state h.
        output_size: The number of expected features in the output y.
        num_layers: Number of recurrent layers. Default: 1
        nonlinearity: The non-linearity to use ['tanh'/'relu'/'id'].
            Default: 'tanh'
        batch_first: If ``True``, then the input and output tensors are provided
            as (batch, seq, feature). Default: ``False``
        leaking_rate: Leaking rate of reservoir's neurons. Default: 1
        spectral_radius: Desired spectral radius of recurrent weight matrix.
            Default: 0.9
        w_ih_scale: Scale factor for first layer's input weights (w_ih_l0). It
            can be a number or a tensor of size '1 + input_size' and first_
            ↪ element
                is the bias' scale factor. Default: 1
        lambda_reg: Ridge regression's shrinkage parameter. Default: 1
        density: Recurrent weight matrix's density. Default: 1
        w_io: If 'True', then the network uses trainable input-to-output
            connections. Default: ``False``
        readout_training: Readout's training algorithm_
            ↪ ['gd'/'svd'/'cholesky'/'inv'].
                If 'gd', gradients are accumulated during backward
                pass. If 'svd', 'cholesky' or 'inv', the network will learn_
            ↪ readout's
                parameters during the forward pass using ridge regression. The
                coefficients are computed using SVD, Cholesky decomposition or
                standard ridge regression formula. 'gd', 'cholesky' and 'inv'
                permit the usage of mini-batches to train the readout.
                If 'inv' and matrix is singular, pseudoinverse is used.
        output_steps: defines how the reservoir's output will be used by ridge
            regression method ['all', 'mean', 'last'].
                If 'all', the entire reservoir output matrix will be used.
                If 'mean', the mean of reservoir output matrix along the timesteps
                dimension will be used.
```

If 'last', only the last timestep of the reservoir output matrix will be used.

'mean' and 'last' are useful for classification tasks.

Inputs: input, washout, h_0, target

input (seq_len, batch, input_size): tensor containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack\_padded\_sequence`

washout (batch): number of initial timesteps during which output of the reservoir is not forwarded to the readout. One value per batch's sample.

h_0 (num_layers, batch, hidden_size): tensor containing the initial reservoir's hidden state for each element in the batch. Defaults to zero if not provided.

target (seq_len*batch - washout*batch, output_size): tensor containing the features of the batch's target sequences rolled out along one axis, minus the washouts and the padded values. It is only needed for readout's training in offline mode. Use `prepare\_target` to compute it.

Outputs: output, h_n

- output (seq_len, batch, hidden_size): tensor containing the output features (h_k) from the readout, for each k.

- **h_n** (num_layers * num_directions, batch, hidden_size): tensor containing the reservoir's hidden state for k=seq_len.

"""

```
def __init__(self, input_size, hidden_size, output_size, num_layers=1,
              nonlinearity='tanh', batch_first=False, leaking_rate=1,
              spectral_radius=0.9, w_ih_scale=1, lambda_reg=0, density=1,
              w_io=False, readout_training='svd', output_steps='all'):
    super(ESN, self).__init__()

    self.input_size = input_size
    self.hidden_size = hidden_size
    self.output_size = output_size
    self.num_layers = num_layers
    if nonlinearity == 'tanh':
        mode = 'RES_TANH'
    elif nonlinearity == 'relu':
        mode = 'RES_RELU'
    elif nonlinearity == 'id':
        mode = 'RES_ID'
    else:
        raise ValueError("Unknown nonlinearity '{}'.format(nonlinearity))
    self.batch_first = batch_first
```

```

self.leaking_rate = leaking_rate
self.spectral_radius = spectral_radius
if type(w_ih_scale) != torch.Tensor:
    self.w_ih_scale = torch.ones(input_size + 1)
    self.w_ih_scale *= w_ih_scale
else:
    self.w_ih_scale = w_ih_scale

self.lambda_reg = lambda_reg
self.density = density
self.w_io = w_io
if readout_training in {'gd', 'svd', 'cholesky', 'inv'}:
    self.readout_training = readout_training
else:
    raise ValueError("Unknown readout training algorithm '{}'.format(
        readout_training))

self.reservoir = Reservoir(mode, input_size, hidden_size, num_layers,
                            leaking_rate, spectral_radius,
                            self.w_ih_scale, density,
                            batch_first=batch_first)

if w_io:
    self.readout = nn.Linear(input_size + hidden_size * num_layers,
                             output_size)
else:
    self.readout = nn.Linear(hidden_size * num_layers, output_size)
if readout_training == 'offline':
    self.readout.weight.requires_grad = False

if output_steps in {'all', 'mean', 'last'}:
    self.output_steps = output_steps
else:
    raise ValueError("Unknown task '{}'.format(
        output_steps))

self.XTX = None
self.XTy = None
self.X = None

def forward(self, input, washout, h_0=None, target=None):
    with torch.no_grad():

        output, hidden = self.reservoir(input, h_0)

        if self.batch_first:
            seq_lengths = output.size(0) * [output.size(1)]

```

```

else:
    seq_lengths = output.size(1) * [output.size(0)]

if self.batch_first:
    output = output.transpose(0, 1)

output, seq_lengths = washout_tensor(output, washout, seq_lengths)

if self.w_io:

    input_lengths = [input.size(0)] * input.size(1)

    if self.batch_first:
        input = input.transpose(0, 1)

    input, _ = washout_tensor(input, washout, input_lengths)
    output = torch.cat([input, output], -1)

if self.readout_training == 'gd' or target is None:
    with torch.enable_grad():
        output = self.readout(output)

        if self.batch_first:
            output = output.transpose(0, 1)

        # Uncomment if you want packed output.
        # if is_packed:
        #     output = pack_padded_sequence(output, seq_lengths,
        #                                     batch_first=self.
↪batch_first)

        return output, hidden

else:
    batch_size = output.size(1)

    X = torch.ones(target.size(0), 1 + output.size(2), ↵
↪device=target.device)
    row = 0
    for s in range(batch_size):
        if self.output_steps == 'all':
            X[row:row + seq_lengths[s], 1:] = output[:
↪seq_lengths[s],
                                                    s]

            row += seq_lengths[s]
        elif self.output_steps == 'mean':
            X[row, 1:] = torch.mean(output[:seq_lengths[s], s], 0)

```

```

        row += 1
    elif self.output_steps == 'last':
        X[row, 1:] = output[seq_lengths[s] - 1, s]
        row += 1

    if self.readout_training == 'cholesky':
        if self.XTX is None:
            self.XTX = torch.mm(X.t(), X)
            self.XTy = torch.mm(X.t(), target)
        else:
            self.XTX += torch.mm(X.t(), X)
            self.XTy += torch.mm(X.t(), target)

    elif self.readout_training == 'svd':
        # Scikit-Learn SVD solver for ridge regression.
        U, s, V = torch.svd(X)
        idx = s > 1e-15 # same default value as scipy.linalg.pinv
        s_nnz = s[idx][:, None]
        UTy = torch.mm(U.t(), target)
        d = torch.zeros(s.size(0), 1, device=X.device)
        d[idx] = s_nnz / (s_nnz ** 2 + self.lambda_reg)
        d_UT_y = d * UTy
        W = torch.mm(V, d_UT_y).t()

        self.readout.bias = nn.Parameter(W[:, 0])
        self.readout.weight = nn.Parameter(W[:, 1:])
    elif self.readout_training == 'inv':
        self.X = X
        if self.XTX is None:
            self.XTX = torch.mm(X.t(), X)
            self.XTy = torch.mm(X.t(), target)
        else:
            self.XTX += torch.mm(X.t(), X)
            self.XTy += torch.mm(X.t(), target)

    return None, None

def fit(self):
    if self.readout_training in {'gd', 'svd'}:
        return

    if self.readout_training == 'cholesky':
        W = torch.linalg.solve(self.XTy,
                               self.XTX + self.lambda_reg * torch.eye(
                                   self.XTX.size(0), device=self.XTX.
↪device))[0].t()
        self.XTX = None

```

```

        self.XTy = None

        self.readout.bias = nn.Parameter(W[:, 0])
        self.readout.weight = nn.Parameter(W[:, 1:])
    elif self.readout_training == 'inv':
        I = (self.lambda_reg * torch.eye(self.XTX.size(0))).to(
            self.XTX.device)
        A = self.XTX + I
        X_rank = torch.linalg.matrix_rank(A).item()

        if X_rank == self.X.size(0):
            W = torch.mm(torch.inverse(A), self.XTy).t()
        else:
            W = torch.mm(torch.pinv(A), self.XTy).t()

        self.readout.bias = nn.Parameter(W[:, 0])
        self.readout.weight = nn.Parameter(W[:, 1:])

        self.XTX = None
        self.XTy = None

    def reset_parameters(self):
        self.reservoir.reset_parameters()
        self.readout.reset_parameters()

```

```

[123]: import numpy as np

def root_relative_squared_error(y_true, y_pred):
    """
    Calculates the Root Relative Squared Error (RRSE).

    Args:
        y_true (array-like): Array of true values.
        y_pred (array-like): Array of predicted values.

    Returns:
        float: RRSE value.
    """
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    # Calculate the mean squared error (MSE)
    mse = np.mean((y_true - y_pred) ** 2)

    # Calculate the squared difference between the actual values and the mean
    ↪ of the actual values
    mean_actual = np.mean(y_true)

```

```
squared_difference = np.mean((y_true - mean_actual) ** 2)

# Calculate the root relative squared error (RRSE)
rrse = np.sqrt(mse / squared_difference)

return rrse
```

Nesta lista de exercícios, iremos avaliar o uso de Echo State Network para previsão de séries temporais (ou identificação de sistemas).

Não esqueça que a ESN é **bastante** dependente da inicialização do modelo, visto que parte dos pesos **não** são aprendidos. Portanto, não esqueça de rodar uma quantidade razoável de vezes o modelo para cada item, de modo a ter uma avaliação mais confiável

```
[21]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

1. Escolha uma base de dados presente em <https://homes.esat.kuleuven.be/~smc/daisy/daisydata.html> (conforme visto em sala de aula) ou no repositório. Faça um gráfico de todas as variáveis do problema para avaliar a dinâmica de cada uma delas.

```
[53]: import torch.nn
import numpy as np
import time

dtype = torch.float
torch.set_default_dtype(dtype)

filename = '/home/thiago/Redes_Neuraais/PUC_Redes_Neuraais/data/pHdata.dat'

data = np.loadtxt(filename, dtype=np.float32)

X_data = np.expand_dims(data[:, :3], axis=1)
Y_data = np.expand_dims(data[:, 3:], axis=1)
```

```
[54]: print(X_data.shape, Y_data.shape)
```

```
(2001, 1, 3) (2001, 1, 1)
```

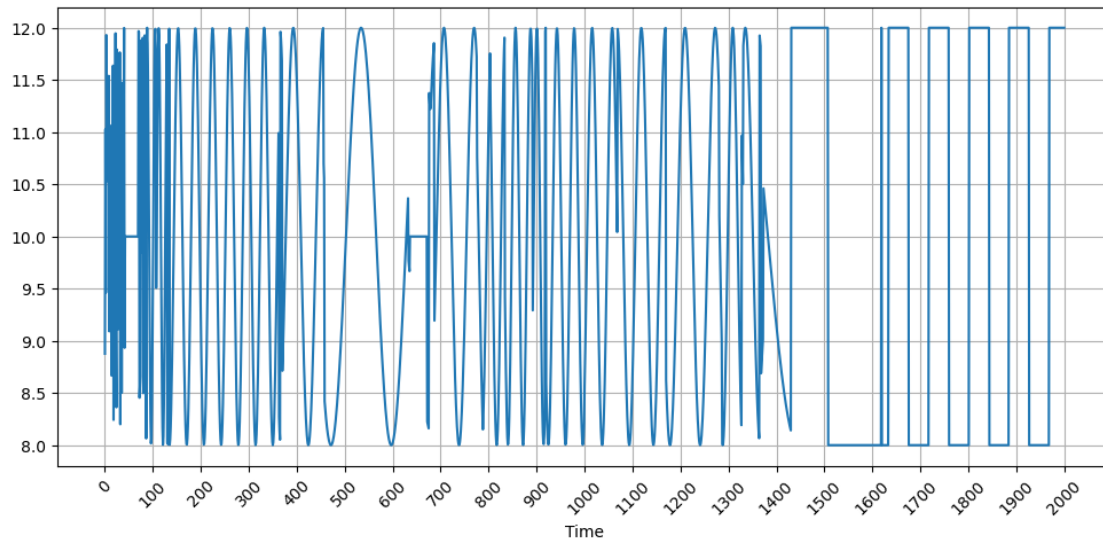
```
[74]: # Acid solution flow in liters
values = X_data[:, 0, 1]
x = np.arange(len(values))
plt.figure(figsize=(10, 5))
plt.xlabel('Time')
plt.xticks(x[::100], rotation=45) # Show every 10th tick for clarity
plt.grid(True)
```



```
plt.tight_layout()

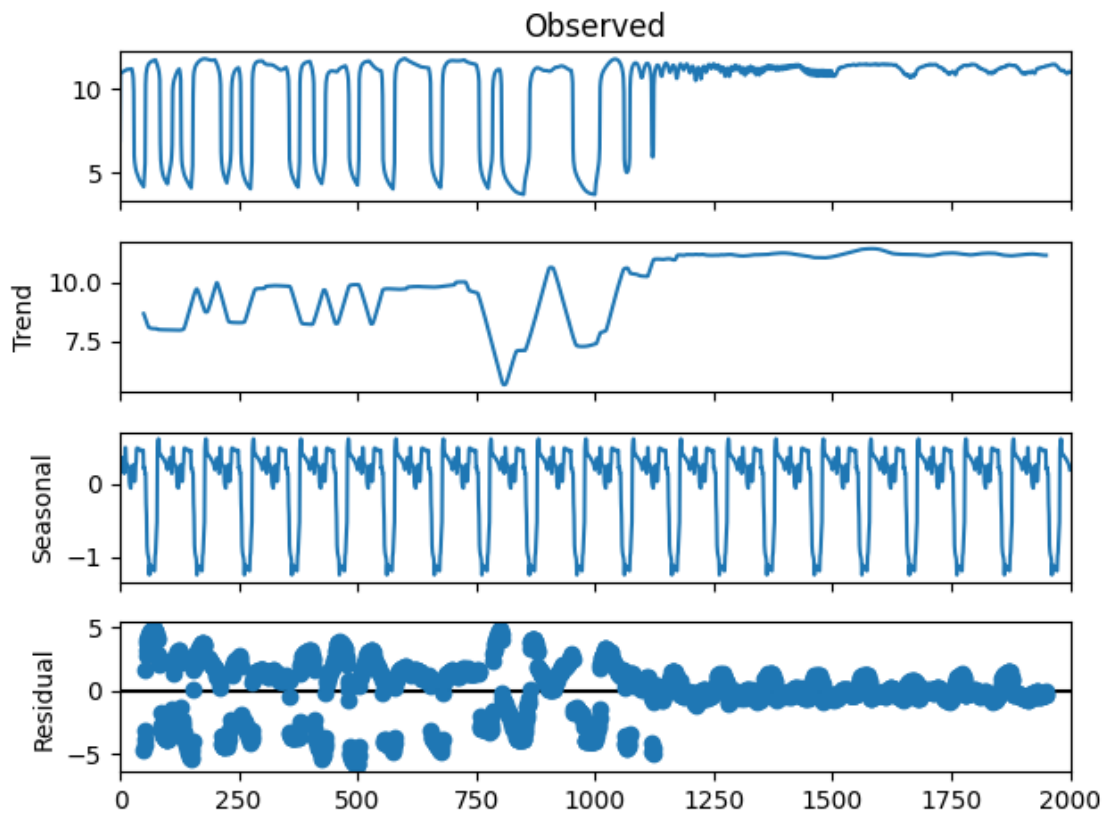
plt.plot(x, values)
```

[74]: [<matplotlib.lines.Line2D at 0x7b34449b8a00>]



```
[88]: result = seasonal_decompose(values, model='additive', period=100)

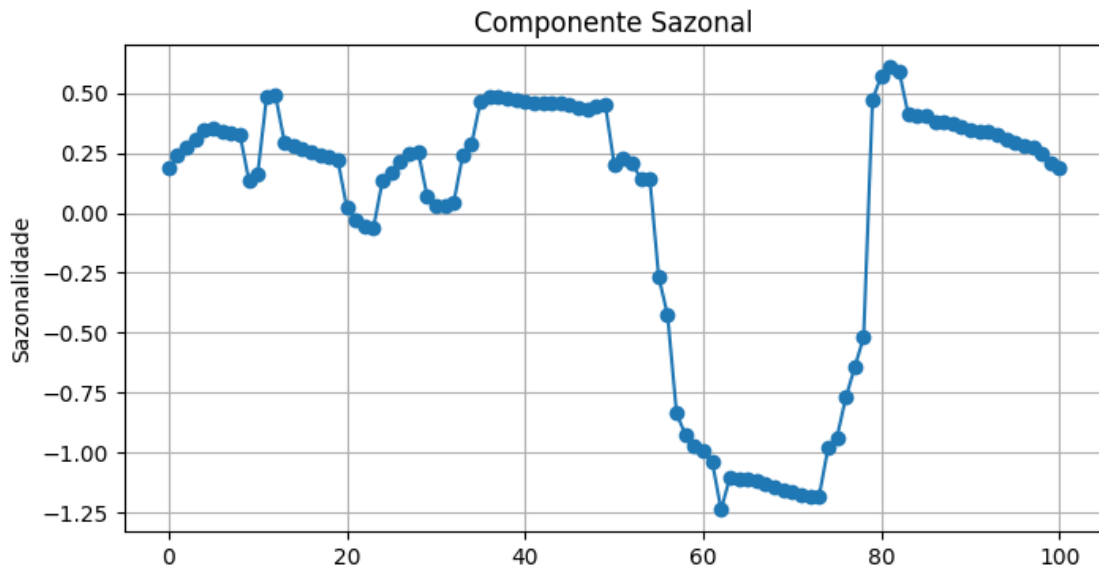
result.plot()
plt.show()
```



```
[89]: seasonal = result.seasonal
seasonal_cycle = seasonal[:101]
# Plot 3: Sazonalidade

time_stamp = np.arange(0, 101)
print(time_stamp.shape, seasonal_cycle.shape)
plt.figure(figsize=(8,4))
plt.plot(time_stamp, seasonal_cycle, marker='o')
plt.title('Componente Sazonal')
plt.ylabel('Sazonalidade')
plt.grid(True)
plt.show()
```

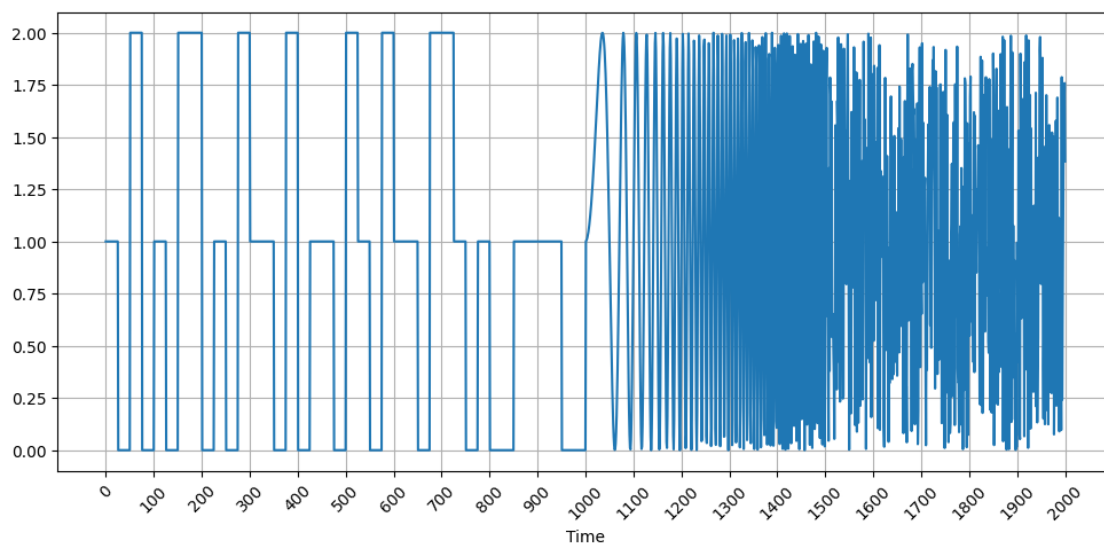
```
(101,) (101,)
```



```
[90]: # Base solution flow in liters
values = X_data[:, 0, 2]
x = np.arange(len(values))
plt.figure(figsize=(10, 5))
plt.xlabel('Time')
plt.xticks(x[::100], rotation=45) # Show every 10th tick for clarity
plt.grid(True)
plt.tight_layout()

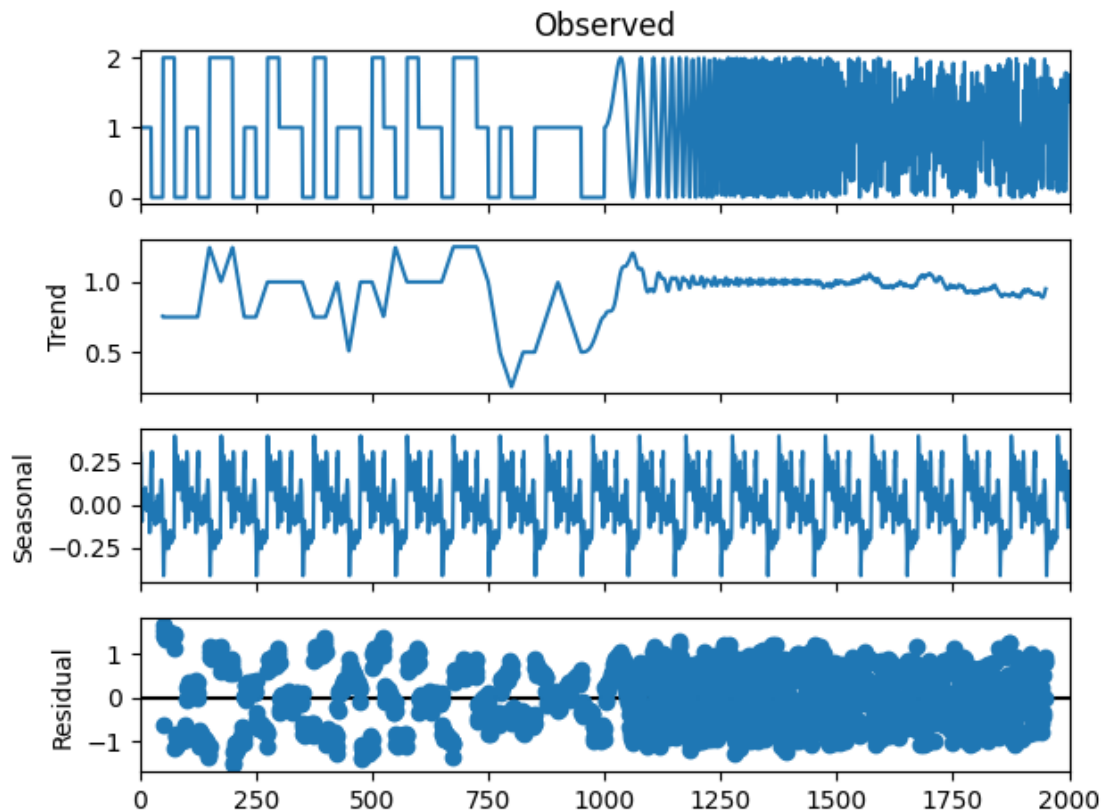
plt.plot(x, values)
```

[90]: [<matplotlib.lines.Line2D at 0x7b34e4b3f280>]



```
[91]: result = seasonal_decompose(values, model='additive', period=100)

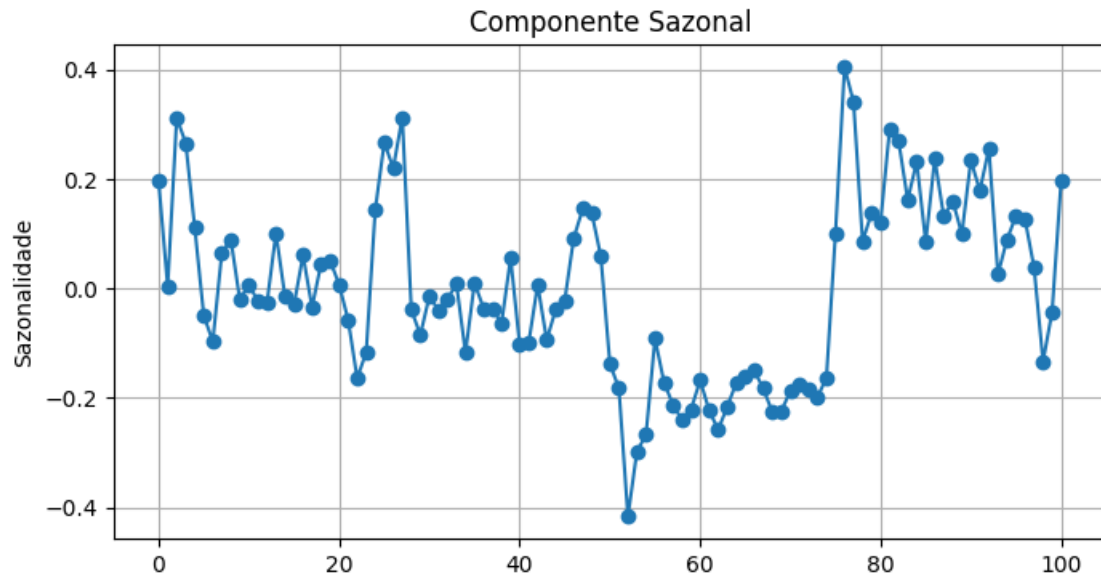
result.plot()
plt.show()
```



```
[92]: seasonal = result.seasonal
seasonal_cycle = seasonal[:101]
# Plot 3: Sazonalidade

time_stamp = np.arange(0, 101)
print(time_stamp.shape, seasonal_cycle.shape)
plt.figure(figsize=(8,4))
plt.plot(time_stamp, seasonal_cycle, marker='o')
plt.title('Componente Sazonal')
plt.ylabel('Sazonalidade')
plt.grid(True)
plt.show()
```

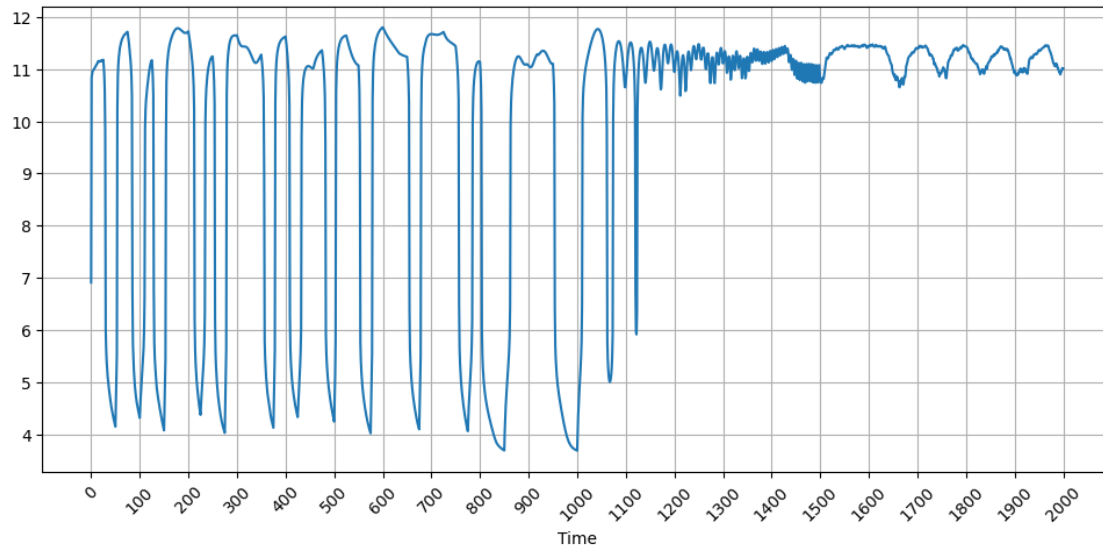
(101,) (101,)



```
[93]: # pH of the solution in the tank
values = Y_data[:, 0, 0]
x = np.arange(len(values))
plt.figure(figsize=(10, 5))
plt.xlabel('Time')
plt.xticks(x[::100], rotation=45) # Show every 10th tick for clarity
plt.grid(True)
plt.tight_layout()

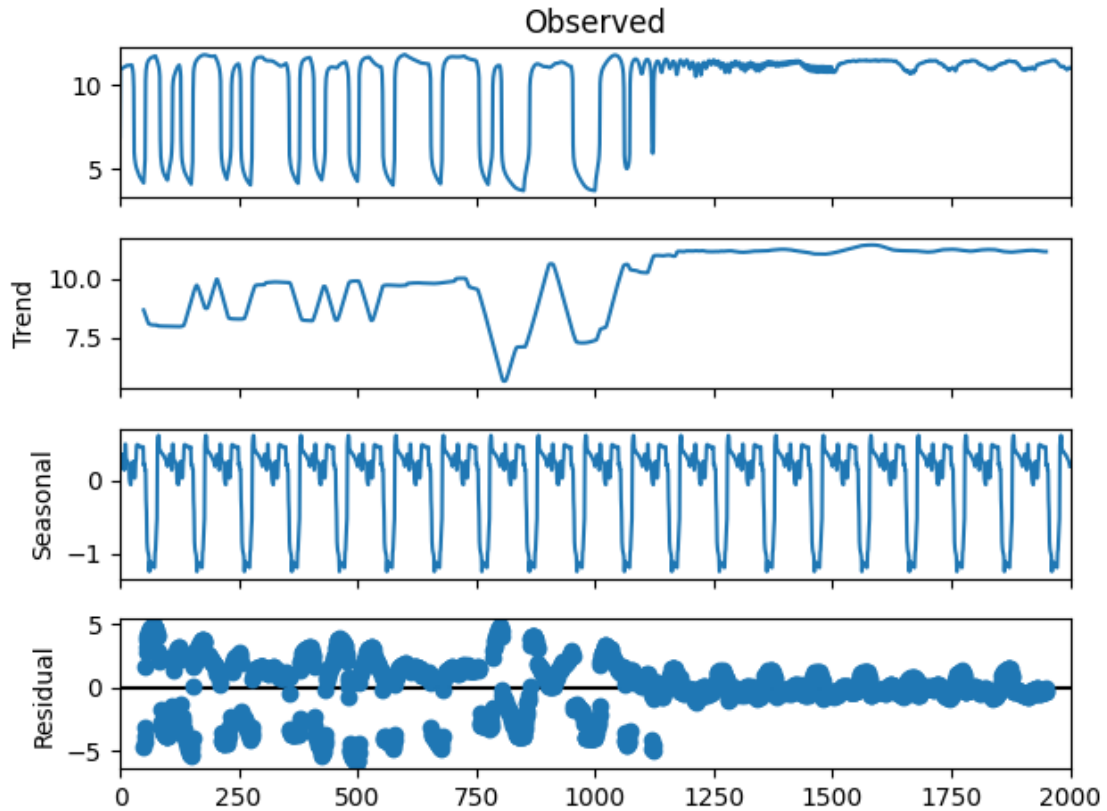
plt.plot(x, values)
```

```
[93]: [<matplotlib.lines.Line2D at 0x7b34435a6770>]
```



```
[94]: result = seasonal_decompose(values, model='additive', period=100)

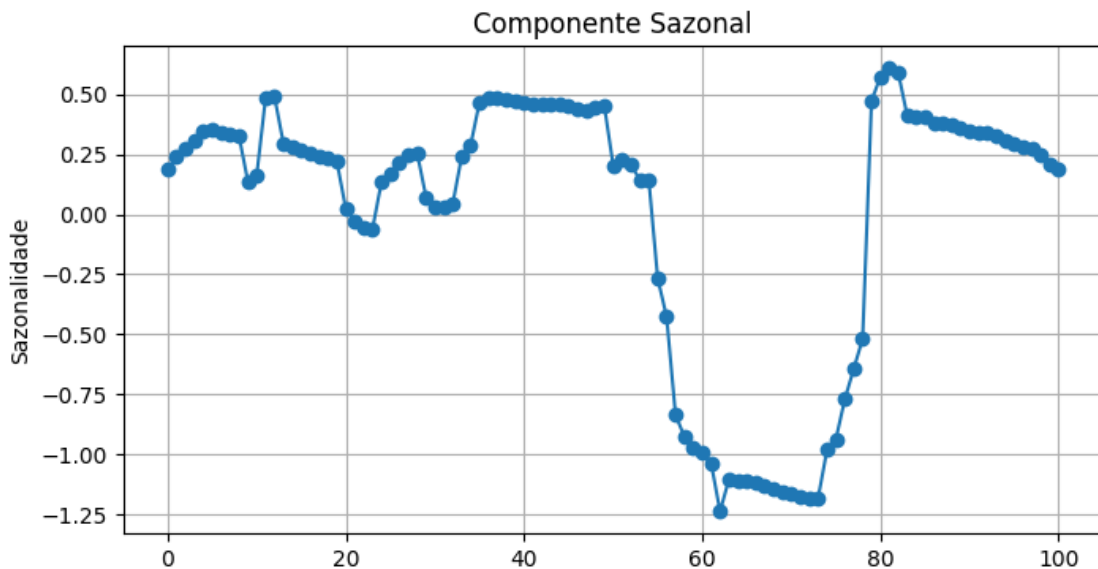
result.plot()
plt.show()
```



```
[95]: seasonal = result.seasonal
seasonal_cycle = seasonal[:101]
# Plot 3: Sazonalidade

time_stamp = np.arange(0, 101)
print(time_stamp.shape, seasonal_cycle.shape)
plt.figure(figsize=(8,4))
plt.plot(time_stamp, seasonal_cycle, marker='o')
plt.title('Componente Sazonal')
plt.ylabel('Sazonalidade')
plt.grid(True)
plt.show()
```

(101,) (101,)



O conjunto de dados é composto de 2 variáveis de entrada, “Acid solution flow in liters” e “Base solution flow in liters”, e uma variável de saída “pH of the solution in the tank”.

A primeira variável “Acid solution flow in liters” apresenta uma sazonalidade com variações sistemáticas e variações bem definidas em um período de 100 segundos. A tendência mostra capturar relações de longo prazo, mas apresenta-se instável em certos trechos com variações abruptas. O ruído também se mostra bem alto corroborando, juntamente com a tendência instável, para a utilização de um modelo não linear que capture essas dinâmicas complexas.

A segunda variável “Base solution flow in liters”, apresenta uma dinâmica com oscilações abruptas quase binárias no começo da série (0 ou 2), enquanto na segunda metade apresenta um aumento da frequência e alta variância. A decomposição mostra que a tendência acompanha essas transições,

ela oscila no início e se estabiliza no final, já que o valor médio permanece aproximadamente o mesmo. O gráfico da sazonalidade por sua vez mostra variações recorrentes mas com amplitude menos expressivas. Os resíduos seguem altos, sobretudo na segunda metade.

A variável target por sua vez, mede o pH da solução de ácido e base. Podemos ver que ela apresenta dois regimes como as duas anteriores. A primeira parte da série mostra uma grande oscilação com periodicidade bem marcada e depois passa a oscilar em torno de um valor mais alta com amplitude próxima dos picos do primeiro regime. A decomposição mostra que a tendência acompanha esse movimento, apresentando quedas e recuperações conforme a média local da série muda, mas estabilizando quando a série se estabiliza em um novo patamar. A componente sazonal é bem evidente, com ciclos regulares e relativamente bem definidos, indicando a presença de um padrão periódico significativo com período 100. Os resíduos estão concentrados nos momentos de maior variação, mas diminuem quando a série se estabiliza.

2. Rode o modelo ESN com uma configuração inicial de hiperparâmetros. Justifique a escolha de cada uma delas.

```
[132]: import torch.nn
import numpy as np
import time

dtype = torch.float
torch.set_default_dtype(dtype)

filename = '/home/thiago/Redes_Neuraais/PUC_Redes_Neuraais/data/pHdata.dat'

data = np.loadtxt(filename, dtype=np.float32)

X_data = np.expand_dims(data[:, 1:3], axis=1)
Y_data = np.expand_dims(np.expand_dims(data[:, 3], axis=1), axis=2)

X_data = torch.from_numpy(X_data)
Y_data = torch.from_numpy(Y_data)
```

```
[133]: print(X_data.shape, Y_data.shape)

torch.Size([2001, 1, 2]) torch.Size([2001, 1, 1])
```

```
[134]: s = int(0.8*X_data.shape[0]) # Usando 80% para treino e 20% para teste

trX = X_data[:s]
trY = Y_data[:s]
tsX = X_data[s:]
tsY = Y_data[s:]
tsY = tsY.reshape(tsY.shape[0], tsY.shape[2])
```

```
[175]: print(trX.shape, tsY.shape)

torch.Size([1600, 1, 2]) torch.Size([401, 1])
```



```

[138]: washout = [100]

# Training
trY_flat = prepare_target(trY.clone(), [trX.size(0)], washout)

[139]: print(trY_flat.shape)

torch.Size([1500, 1])

[140]: loss_fcn = torch.nn.MSELoss()

[ ]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↳ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.9 # Dependência do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

    loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

    loss_test.append(loss_fcn(output_, tsY).item())

    output = torch.squeeze(output_.detach(), dim=1)
    rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
    mae_.append(np.array(torch.mean(abs(output - tsY), dim=0)))

loss_train = np.array(loss_train)

```

```

loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
    ↳ {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0360 +- 0.0602, MAE = 2.3747 +- 1.6184

Os hiperparâmetros iniciais foram escolhidos de tal maneira que fizesse sentido para o problema do dataset em questão. O tamanho do reservatório deve ser escolhido de maneira que eles fiquem entre $T/10$ e $T/2$, sendo $T = 2001$. Visto que a série temporal das concentrações é bem periódica e o problema não é muito complexo devido a quantidade baixa de variáveis, foi escolhido o tamanho mínimo para o reservatório. Como a matriz deve ser bem esparsa, foi escolhido a densidade com 60% dos neurônios do reservatório. Para o raio espectral foi escolhido um valor grande de maneira que o aprendizado fosse lento. O leaking rate foi escolhido com um valor alto para o estado atual ter uma grande dependência do estado anterior.

3. Modifique o tamanho do reservatório e avalie o resultado dessa mudança. De que forma a alteração dos pesos pode impactar no resultado final?

```

[159]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 500 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
    ↳ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.9 # Dependencia do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
        ↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

```

```

loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
    ↳ {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0096 +- 0.0164, MAE = 6.5020 +- 6.1617

```

[160]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 1000 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
    ↳ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.9 # Dependência do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
        ↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

    loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

```

```

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↪ = {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0077 +- 0.0115, MAE = 5.4341 +- 1.5215

O tamanho do reservatório é responsável pela capacidade de aprendizado dos neurônios. É necessário definir a quantidade de neurônios para a complexidade do problema. Uma quantidade alta de neurônios pode acarretar em overfitting, fazendo com que os neurônios se ajustem demais aos dados, como é o caso dos experimentos feitos com 500 e 1000 neurônios, os quais tiveram desempenho pior no conjunto de teste que o experimento com 200 neurônios.

4. Modifique o parâmetro de **leaking rate** e avalie o resultado dessa mudança. De que forma a alteração desse hiperparâmetro pode impactar no resultado final?

```

[161]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↪ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.6 # Dependência do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↪ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

```

```

# Test
output_, hidden = model(tsX, [0], hidden)

loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↳ {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0106 +- 0.0442, MAE = 1.5316 +- 0.6335

```

[162]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↳ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.3 # Dependencia do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

# Test

```

```

output_, hidden = model(tsX, [0], hidden)

loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↪ = {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 0.9958 +- 0.0634, MAE = 1.1600 +- 0.6197

```

[163]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↪ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.1 # Dependência do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↪ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

```

```

loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
    ↳ {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')
```

Output #1 - RRSE = 1.0351 +- 0.0974, MAE = 0.9784 +- 0.2796

O leaking rate é responsável pela dependência do estado atual para o estado passado. Com um leaking rate alto, a rede dá mais contribuição para o estado atual e com o leaking rate baixo dá mais contribuição para o leaking rate baixo. Pelos experimentos podemos perceber que os erros foram mais baixos para um valor menor do leaking rate, mostrando que para esse problema é mais importante saber a informação passada armazenada do que a atual.

5. Modifique o parâmetro de **esparsidade** e avalie o resultado dessa mudança. De que forma a alteração do volume de conexões pode impactar no resultado final?

```

[164]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
    ↳ amostras.  $T = 2001$ 
density = 0.9
leaking_rate = 0.1 # Dependencia do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
    ↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
```

```

output_train, hidden = model(trX, washout)

# Test
output_, hidden = model(tsX, [0], hidden)

loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↪ = {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0779 +- 0.1000, MAE = 0.7782 +- 0.1530

```

[165]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↪ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.1 # Dependencia do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↪ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

```



```

# Test
output_, hidden = model(tsX, [0], hidden)

loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↳ {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0247 +- 0.0377, MAE = 0.7702 +- 0.1662

```

[166]: n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↳ amostras.  $T = 2001$ 
density = 0.3
leaking_rate = 0.1 # Dependencia do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

# Test

```

```

output_, hidden = model(tsX, [0], hidden)

loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

loss_test.append(loss_fcn(output_, tsY).item())

output = torch.squeeze(output_.detach(), dim=1)
rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↪ = {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0560 +- 0.1023, MAE = 0.9240 +- 0.3593

Foi feito 3 experimentos com densidades de neurônios diferentes. O melhor resultado foi o intermediário com 40% dos neurônios esparsos. O resultado com uma menor esparsidade não foi o melhor apesar da maior quantidade de parâmetros, pois a esparsidade obriga a rede a aprender dinâmicas pouco correlacionadas que acabam se complementando. Entretanto, se eles ficarem muito esparsos não conseguirão aprender dinâmicas não lineares complexas. Isso explique porque o resultado com valor de esparsidade intermediário foi melhor.

6. Modifique o parâmetro de **tempo de washout** e avalie o resultado dessa mudança. De que forma esta pode impactar no resultado final?

```

[169]: washout = [500]

# Training
trY_flat = prepare_target(trY.clone(), [trX.size(0)], washout)

n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↪ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.1 # Dependência do estado atual para o anterior
spectral_radius = 0.9

```

```

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
    ↪leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

    loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

    loss_test.append(loss_fcn(output_, tsY).item())

    output = torch.squeeze(output_.detach(), dim=1)
    rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
    mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
    ↪= {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0905 +- 0.1282, MAE = 1.4970 +- 0.5701

```

[170]: washout = [250]

# Training
trY_flat = prepare_target(trY.clone(), [trX.size(0)], washout)

n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
    ↪amostras.  $T = 2001$ 

```

```

density = 0.6
leaking_rate = 0.1 # Dependencia do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
    ↪leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

    loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

    loss_test.append(loss_fcn(output_, tsY).item())

    output = torch.squeeze(output_.detach(), dim=1)
    rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
    mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↪= {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.1340 +- 0.1331, MAE = 1.5983 +- 0.7113

```

[171]: washout = [100]

# Training
trY_flat = prepare_target(trY.clone(), [trX.size(0)], washout)

n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []

```

```

mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↳ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.1 # Dependência do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

    loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

    loss_test.append(loss_fcn(output_, tsY).item())

    output = torch.squeeze(output_.detach(), dim=1)
    rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
    mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↳ = {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0286 +- 0.0662, MAE = 0.8084 +- 0.2439

```

[172]: washout = [50]

# Training
trY_flat = prepare_target(trY.clone(), [trX.size(0)], washout)

n = 10
input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []

```

```

loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↳ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.1 # Dependência do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

    loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

    loss_test.append(loss_fcn(output_, tsY).item())

    output = torch.squeeze(output_.detach(), dim=1)
    rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
    mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↳ = {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0791 +- 0.1021, MAE = 1.0891 +- 0.2642

```

[173]: washout = [10]

# Training
trY_flat = prepare_target(trY.clone(), [trX.size(0)], washout)

n = 10

```

```

input_size = trX.shape[2]
output_size = trY.shape[2]
loss_train = []
loss_test = []
rrse_v1 = []
rrse_v2 = []
mae_ = []
hidden_size = 200 # Deve ser entre  $T/10 < N < T/2$ , onde  $T$  é o número de
↳ amostras.  $T = 2001$ 
density = 0.6
leaking_rate = 0.1 # Dependência do estado atual para o anterior
spectral_radius = 0.9

for i in range(n):
    model = ESN(input_size, hidden_size, output_size, density=density,
↳ leaking_rate=leaking_rate, spectral_radius=spectral_radius)

    _ = model(trX, washout, None, trY_flat)

    model.fit()
    output_train, hidden = model(trX, washout)

    # Test
    output_, hidden = model(tsX, [0], hidden)

    loss_train.append(loss_fcn(output_train, trY[washout[0]:]).item())

    loss_test.append(loss_fcn(output_, tsY).item())

    output = torch.squeeze(output_.detach(), dim=1)
    rrse_v1.append(root_relative_squared_error(output[:,0], tsY[:,0]))
    mae_.append(np.array(torch.mean(abs(output - tsY),dim=0)))

loss_train = np.array(loss_train)
loss_test = np.array(loss_test)
rrse_v1 = np.array(rrse_v1)
mae_ = np.array(mae_)

print(f'Output #1 - RRSE = {np.mean(rrse_v1):.4f} +- {np.std(rrse_v1):.4f}, MAE_
↳ = {np.mean(mae_,axis=0)[0]:.4f} +- {np.std(mae_,axis=0)[0]:.4f}')

```

Output #1 - RRSE = 1.0036 +- 0.0623, MAE = 1.0480 +- 0.4504

O washout é o momento de aquecimento do reservatório. Durante ele, são atualizados os estados da rede para que ela esqueça os estados provenientes da inicialização aleatória. Como o novo estado atual, depende do estado anterior somado a contribuição de todos os estados anteriores, é necessário várias iterações para que o histórico se atualize e a ponderação dos estados antigos $(1-\text{leaking_rate})^k$

tenha uma influência menor nos novos estados. Isso explica porque os experimentos de washout de 100 foi melhor que o de 50 e 10. Entretanto, um washout grande demais também é um problema, pois os dados utilizados no washout são jogados fora e não serão utilizados no treinamento para atualizar a camda de saída da ESN, perdendo assim informações importantes do conjunto de dados. Isso explica porque os washouts de 250 e 500 foram pior que o de 100, já que nossa base de dados continha 1600 amostras no conjunto de dados apenas.